

INFORME ANALITICO TECNICO

SEMÁFOROS

GRUPO N°1



Integrantes:

Lezcano, Axel Antonio

Cardozo, Fabricio Hernan

Almirón, Franco Gonzalo

Fundamentos De Diseño

Fundamentos sobre el diseño funcional adoptado:

1) Transición (Implementación final):

```
;;correcto:
(defun transicion (color-actual cambiar-a)
  (cond
    ((and (eql color-actual 'en-verde-intermitente) (eql cambiar-a 'cambiar-a-amarillo)) (list color-actual "CAMBIAR-A-AMARILLO" ))
    ((and (eql color-actual 'en-amarillo-intermitente) (eql cambiar-a 'cambiar-a-rojo)) (list color-actual "CAMBIAR-A-ROJO"))
    ((and (eql color-actual 'en-rojo-intermitente) (eql cambiar-a 'cambiar-a-verde)) (list color-actual "CAMBIAR-A-VERDE" ))
    ((and (eql color-actual 'en-verde) (eql cambiar-a 'cambiar-a-verde-intermitente)) (list color-actual "CAMBIAR-A-VERDE-INTERMITENTE"))
    ((and (eql color-actual 'en-amarillo) (eql cambiar-a 'cambiar-a-amarillo-intermitente)) (list color-actual "CAMBIAR-A-AMARILLO-INTERMITENTE"))
    ((and (eql color-actual 'en-rojo) (eql cambiar-a 'cambiar-a-rojo-intermitente)) (list color-actual "CAMBIAR-A-ROJO-INTERMITENTE"))
    (t (list color-actual 'accion-por-defecto))
  )
)
```

La lógica utilizada en el siguiente código y la forma en la cual fue implementada de la siguiente forma: para la modelación de los distintos cambios del semáforo (teniendo en cuenta las entradas por parámetro asignadas al comienzo "color-actual" y "cambiar-a") se utilizó una estructura de alternativa múltiple (cond), donde cada caso representaba un caso posible y válido del ciclo (rojo→verde→amarillo), el cual mediante un operador lógico *and* se fue verificando que cumpliera justamente ese mismo cambio. Luego con la llegada de la intermitencia por cambios de colores se actualizó la implementación de la función, colocando 3 nuevos casos, cada uno verificando si el cambio del color original (sea un color o la intermitencia del mismo) pasa a su siguiente estado verídico.

2) Timer (implementación final)

```
(defun timer (timestamp)
  (cond
    ;(89 92 212 215 221 224)
    ((<= 0 (mod timestamp 225) 89) 'en-rojo )
    ((<= 90 (mod timestamp 225) 92) 'en-rojo-intermitente )
    ((<= 93 (mod timestamp 225) 212) 'en-verde )
    ((<= 213 (mod timestamp 225) 215) 'en-verde-intermitente )
    ((<= 216 (mod timestamp 225) 221) 'en-amarillo )
    (t 'en-amarillo-intermitente)
  )
)
```

La lógica implementada en este requerimiento fue de la siguiente manera: al principio con únicamente 4 tipos de casos distintos respecto a los colores (rojo, verde y amarillo), se calculaba el resto del tiempo Unix, el cual accedía a la función mediante el parámetro, para calcular el tiempo exacto en donde estábamos parados en ese último ciclo, comenzando desde el 0 hasta 215 que eran todos los casos posibles, siendo el 0 el segundo 1 del ciclo y 215 el segundo 216. De esta forma los intervalos de números entre ellos representaban el segundo (n+1) del ciclo. Luego con la nueva implementación con la llegada de la intermitencia, agregamos 3 segundos a cada transición de color del semáforo, quedando así, un ciclo de 225 segundos (9 segundos más para las intermitencias).

3) loginLights (implementación final):

```
(defun login-lights (color-actual cambio-color unix-temp)
  (format t "~% Tiempo ~A la luz ah cambiado de color ~A a ~A-%"
    (local-time:format-timestring nil (local-time:unix-to-timestamp unix-temp):format '(:year 4) "-" (:month 2) "-" (:day 2) " " (:hour 2) ":" (:min 2) ":" (:sec 2))
    (first (transicion color-actual cambio-color)) (second (transicion color-actual cambio-color)))
  )
)
```

Hubo bastante discusión con esta función.

Para el procedimiento de esta función de auditoría, al principio imprimimos en pantalla el tiempo (utilizando get-universal) y el cambio del color-actual al cambiar-color. Luego actualizando la función con local-time, se obtuvo una mejor representación sobre el tiempo, siendo más legible para el lenguaje humano y utilizando la función transición como una forma de asegurar de que el cambio el cual se va a realizar es el correcto, además un cambio adicional en el que en vez de llamar la función para que inserte el tiempo actual al momento de ejecutar esa función, decidimos agregar un parámetro a la función, recibiendo el tiempo unix, siendo más precisos con el tiempo registrado.

4.a) duracion-ciclo (implementación final):

```
(defun duracion-ciclo (rojo verde amarillo)
  (+ rojo verde amarillo 9) ;9 de intermitencia
)
```

En esta parte, según lo comprendido por el grupo, se entendió que la duración del ciclo total va a variar dependiendo de los colores con los que se quiera trabajar en el semáforo. Es una función la cual ayuda a saber cuánto va a durar un ciclo según los distintos colores, teniendo en cuenta siempre que la intermitencia es fija y es de 3 segundos.

4.b) Recomendacion-ciclo (implementacion final):

```
(defun recomendacion-ciclo (duracion)
  (if (and (>= duracion 35) (<= duracion 150))
      "ciclo optimo"
      "ciclo no optimo")
)
```

Esta función se encarga a través de una condicional simple y un operador lógico *and* si la duración del ciclo se encuentra entre los rangos 35 hasta 150. Teniendo esta función como dato de entrada en su parámetro a la función *duracion-ciclo* la cual se habló anteriormente y se encargará de sumar y calcular si está o no en el rango óptimo.

5) ciclos-por-tiempo(implementación final):

```
(defun ciclos-por-tiempo (minutos)
  (print "La cantidad de ciclos es de:")
  (print (truncate (/ (* minutos 60) 225))));truncate toma el resultado de una operacion y elimina el decimal, si el resultado es 28.9, quedaria 28
)
```

En esta función se imprime por pantalla la cantidad de ciclos completos los cuales pueden ser almacenados a la cantidad de minutos la cual es llegada como parámetro, la estrategia

fue simple y sencilla: multiplicar la cantidad de minutos por 60 para transformarlos a la unidad de segundos, luego dividimos ese tiempo por 216 (cantidad de segundos que ocupa un ciclo) y lo truncamos, esto debido que queremos la cantidad de ciclos enteros. Luego se lo modifico a 225 segundos debido a la implementacion de intermitencia.

6) *calcular-Resto-Ini (Función Auxiliar):*

```
(defun calcular-resto-ini (resto-ini)
  (cond ; (89 92 212 215 221 224)
    ((<= 0 resto-ini 89) (list (- 90 resto-ini) 3 120 3 6 3)) ; (- 90 resto-ini) --> indica lo consumido por rojo
    ((<= 89 resto-ini 92) (list 0 (- 93 resto-ini) 120 3 6 3)) ; (- 93 resto-ini) --> indica lo consumido por el rojo-intermitente
    ((<= 93 resto-ini 212) (list 0 0 (- 213 resto-ini) 3 6 3)) ; (- 213 resto-ini) --> indica lo consumido por verde
    ((<= 213 resto-ini 215) (list 0 0 0 (- 216 resto-ini) 6 3)) ; (- 216 resto-ini) --> indica lo consumido por verde-intermitente
    ((<= 216 resto-ini 221) (list 0 0 0 0 (- 222 resto-ini) 3)) ; (- 222 resto-ini) --> indica lo consumido por amarillo
    (t (list 0 0 0 0 0 (- 225 resto-ini))) ; (-225 resto-ini) --> indica lo consumido por amarillo-intermitente
  )
)
```

En esta función, se basó en los cálculos obtenidos de anteriormente antes de ser introducidas las intermitencias. En resumen sobre los cálculos de esta función, se devuelve una lista para poder retornar las cantidades exactas de los segundos consumidos según cada caso. Al ser una función la cual le llegaba como parámetros el resto inicial del tiempo unix (explicada más adelante) nos daba el segundo actual en donde estábamos parados en ese intervalo de tiempo (pudiendo ser del 0-215), tomando ese tiempo se calculaba cuánto tiempo consumido hay desde ese instante hasta el fin de ese ciclo (para el cálculo de porcentajes más adelante). Luego, ahora con la llegada de la intermitencia, siendo los colores de intermitencia independientes de otros colores, se tuvo que construir 3 casos más y además se tuvieron que agregar la intermitencia por separada a la lista retornada.

6.1) *calcular-Resto-Fin (Función Auxiliar):*

```
(defun calcular-resto-fin (resto-fin)
  (cond ; (89 92 212 215 221 224)
    ; Al final de la hora es al revés: calculamos cuánto se consumió desde el inicio de ese último ciclo incompleto
    ; hasta llegar al segundo 'resto-fin'
    ((<= 0 resto-fin 89) (list resto-fin 0 0 0 0 0))
    ; resto-fin --> indica lo consumido por rojo
    ((<= 90 resto-fin 92) (list 90 (- resto-fin 90) 0 0 0 0))
    ; (- resto-fin 90) --> indica lo consumido por el rojo-intermitente
    ((<= 93 resto-fin 212) (list 90 3 (- resto-fin 93) 0 0 0))
    ; (- resto-fin 93) --> indica lo consumido por verde
    ((<= 213 resto-fin 215) (list 90 3 120 (- resto-fin 213) 0 0))
    ; (- resto-fin 213) --> indica lo consumido por verde-intermitente
    ((<= 216 resto-fin 221) (list 90 3 120 3 (- resto-fin 216) 0))
    ; (- resto-fin 221) --> indica lo consumido por amarillo
    (t (list 90 3 120 3 6 (- resto-fin 222)))
    ; (- resto-fin 222) --> indica lo consumido por amarillo-intermitente
  )
)
```

Similar a la función anterior, pero en este caso el parámetro restoFin es el resto de la hora unix + 3600 (la siguiente hora). la única diferencia entre esta y la anterior, es que *calcular-Resto-Fin* devuelve una lista donde los consumidos ya no están adelante, si no desde el resto parado hacia el comienzo de ese ciclo (ya no hacia el final).

6.2) calcular-Portcentajes (Función Auxiliar):

```
(defun calcular-Portcentajes (lista-ini lista-fin)
  (list ; todo basado en la regla de 3 simples
    (float (/ (* (+ 1350 (first lista-fin) (first lista-ini)) 100) 3600)) ; rojo
    (float (/ (* (+ 45 (second lista-fin) (second lista-ini)) 100) 3600)) ; rojo-intermitente
    (float (/ (* (+ 1800 (third lista-fin) (third lista-ini)) 100) 3600)) ; verde
    (float (/ (* (+ 45 (fourth lista-fin) (fourth lista-ini)) 100) 3600)) ; verde-intermitente
    (float (/ (* (+ 90 (fifth lista-fin) (fifth lista-ini)) 100) 3600)) ; amarillo
    (float (/ (* (+ 45 (sixth lista-fin) (sixth lista-ini)) 100) 3600)) ; amarillo-intermitente
  )
)
```

Se aplica el cálculo de la regla de 3 simples, accediendo a cada elemento con los first, second, third, etc, (los elementos se muestran a la derecha en forma de comentarios) y pasandolos a float en el debido caso que los resultados dieran con coma. Entrando en un analisis mas profundo sobre el diseño de esta funcion, se llego a la conclusion que al entrar en 3600 segundos 16 ciclos exactos, si entramos a calcular el porcentaje del tiempo justo entrando a la mitad de un ciclo incompleto, no podemos saber con exactitud cuanto falto ni cuanto se consumo, pero lo que sabemos, es que 15 ciclos entraron asegurados, por ende hacemos el producto de cada color e intermitencia por 15, luego para calcular el ciclo restante se realiza la la suma de los restos del inicio del ciclo y del final.

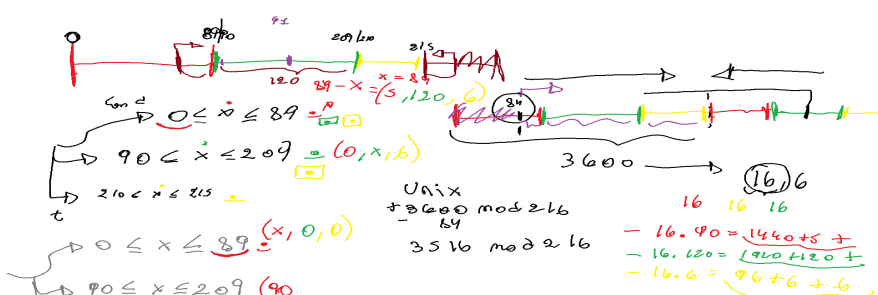
Cabe aclarar que se hizo de esta forma ya que se tomo del diseño anterior de cuando se calculaba el porcentaje con el ciclo de 216 segundos, tranquilamente se podria simplificar mucho más para el caso de la intermitencia.

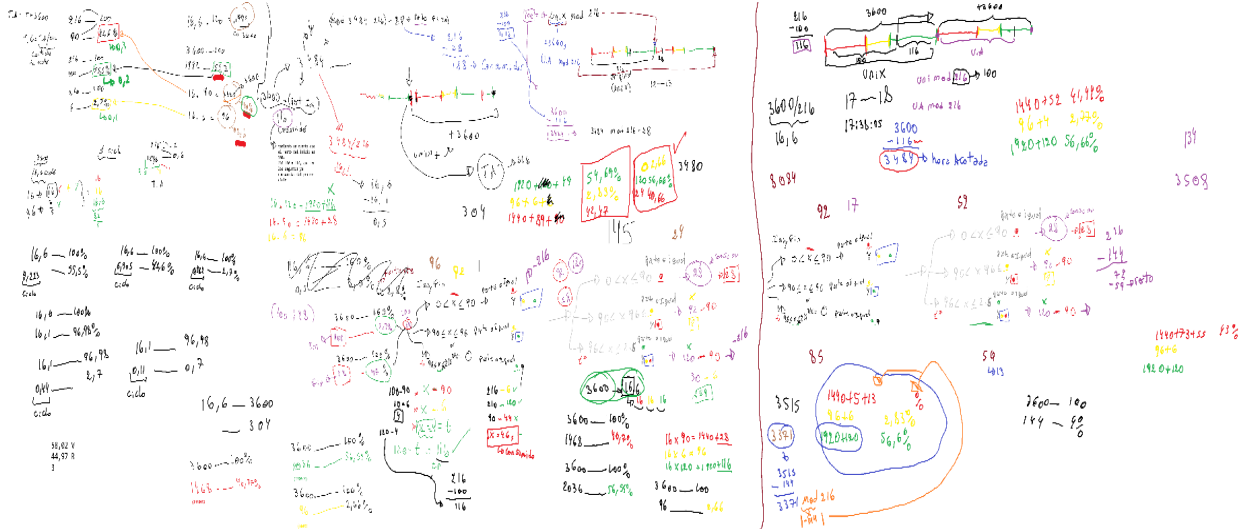
Por otra parte, se puede observar que los datos del parámetro son 2 listas, las cuales corresponden al retorno de las 2 últimas funciones auxiliares que vimos (calcularRestoIni y CalcularRestoFin).

6.3) distribucion-time (Implementación Final):

```
(defun distribucion-Time (unix)
  (if (zerop (mod unix 225)) "| 40.0% rojo| 1.33% rojo-intermitente | 53.33% verde| 1.33% verde-intermitente| 2.66% amarillo| 1.33% amarillo intermitente|"
      (format nil "~%| ~A% rojo| ~A% rojo-intermitente| ~A% verde | ~A% verde-intermitente| ~A% amarillo| ~A% amarillo-intermitente|"
        (first (calcular-Portcentajes (calcular-Resto-Ini (mod unix 225)) (Calcular-Resto-Fin (mod (- 3600 (- 225 (mod unix 225))) 225))))
        (second (calcular-Portcentajes (calcular-Resto-Ini (mod unix 225)) (Calcular-Resto-Fin (mod (- 3600 (- 225 (mod unix 225))) 225))))
        (third (calcular-Portcentajes (calcular-Resto-Ini (mod unix 225)) (Calcular-Resto-Fin (mod (- 3600 (- 225 (mod unix 225))) 225))))
        (fourth (calcular-Portcentajes (calcular-Resto-Ini (mod unix 225)) (Calcular-Resto-Fin (mod (- 3600 (- 225 (mod unix 225))) 225))))
        (fifth (calcular-Portcentajes (calcular-Resto-Ini (mod unix 225)) (Calcular-Resto-Fin (mod (- 3600 (- 225 (mod unix 225))) 225))))
        (sixth (calcular-Portcentajes (calcular-Resto-Ini (mod unix 225)) (Calcular-Resto-Fin (mod (- 3600 (- 225 (mod unix 225))) 225))))
      )
  )
```

Esta función no hace más que imprimir los porcentajes retornados por la función *calcular-Portcentajes*, los cuales a su vez reciben a *Calcular-Resto-Ini* y *Calcular-Resto-Fin*. A continuación se mostrará los cálculos y pensamientos que ayudaron en un principio a la resolución del problema (pero más utilizado como C.A, o también conocidos como cálculos auxiliares).





7) Informe (Implementación Final):

```
(defun informe (color-actual cambio-color unixtemp)
  (crear-informe)
  (with-open-file (stream "informe-ejecucion-semaforo.txt" :direction :output :if-exists :append :if-does-not-exist :create)
    (format stream "~%=====~%")
    (format stream "~A transicion: ~A --> ~A~%"
      (local-time:format-timestring nil (local-time:unix-to-timestamp unixtemp) :format '(:year 4) "-" (:month 2) "-" (:day 2) " " (:hour 2) ":" (:min 2) ":" (:sec 2)))
      (car (transicion color-actual cambio-color)) (caddr (transicion color-actual cambio-color)) )
    )
  ;pasandole login para ver en pantalla
  (loginLights color-actual cambio-color unixtemp)
)
```

La función informe se encarga de escribir en un archivo txt el cambio de color que ocurre en un determinado tiempo, el cual está expresado de una forma fácilmente entendible para el humano.

En primer lugar la función abre el archivo primero en la función (crear-informe), y luego lo hace en esta función. Después, con los format stream que aparecen le dan la instrucción al sistema que escriba dentro del archivo txt los distintos string que aparecen adjuntos al format. Por consiguiente volvemos a toparnos con un format que se encarga de imprimir en el archivo la hora recibida del parámetro unixtemp con el **local-time**, se le indica que queremos que el tiempo sea impreso en formato string le damos el tiempo en "crudo" el tiempo, luego de esto le indicamos cuantos caracteres queremos que se escriba de cada tipo (ejemplo, para los años le indicamos que queremos 4 "2026" por ejemplo, los meses 2 "06" días "14" hora "17" minutos "30" segundos "50". Luego de todo eso se indica el primer elemento de la lista de transición como una forma de asegurar que el valor es el indicado y el último elemento. Al final, luego de que se haya escrito la información el archivo.txt, se llama a la función loggings para mostrar al usuario lo que se escribió.

7.a) crear-informe (Implementación Final):

```
(defun crear-informe ()
  (unless (probe-file "informe-ejecucion-semaforo.txt") ;probe-file comprueba si existe el archivo.txt, si existe, da verdadero, sino falso, y un unless es como if,
    ;pero solo ejecuta si la condición es falsa
    (with-open-file (stream "informe-ejecucion-semaforo.txt" :direction :output)
      (format stream "Informe de Ejecución del Sistema Semafórico-%")
    )
  )
)
```

La función sirve para imprimir el texto de “Informe de ejecución del Sistema semafórico” una única vez al principio del archivo.txt

7.b) cerrar-informe (Implementación Final):

```
(defun cerrar-informe()
  (with-open-file (stream "informe-ejecucion-semaforo.txt":direction :output :if-exists :append)
    (format stream "~% --- Fin del Informe ----~%")
  )
)
```

La función se encarga de que una vez que hayan finalizado todas las llamadas de informe, se escriba un texto al final que diga “Fin del informe”

Bitacora De Bugs

Bitácora de debug de depuración:

Commit fc95dc7:

```
295 + -----|#
296 + (defun cerrar-informe()
297 +   (with-open-file (stream "informe-ejecucion-semaforo.txt":direction :output :if-exists :append)
298 +     (format stream "~% --- Fin del Informe ---%")
299 +   )
300 + )
301 +
290 302 #|-----
291 303 FUNCION: informe
292 304 NATURALEZA: Impura (escribe en pantalla y en un archivo)
293 305 ESTRATEGIA: Secuencial
294 306 IMPACTO: no destructiva
295 307 -----|#
296 308
297 - (defun informe (color-actual cambio-color)
309 + (defun informe (color-actual cambio-color unixtemp)
298 310 (crear-informe)
299 311 (with-open-file (stream "informe-ejecucion-semaforo.txt" :direction :output :if-exists :append :if-does-not-exist :create)
300 312 (format stream "~%=====~%")
301 313 - (format stream "~A: transicion: ~A --> ~A"
302 314 - (local-time:format-timestring nil (local-time:now) :format '(:year 4) "-" (:month 2) "-" (:day 2) " " (:hour 2) ":" (:min 2)))
313 + (format stream "~A: transicion: ~A --> ~A%~"
314 + (local-time:format-timestring nil (local-time:unix-to-timestamp unixtemp) :format '(:year 4) "-" (:month 2) "-" (:day 2) " " (:hour 2) ":" (:min 2)))
303 315 (car (transicion color-actual cambio-color)) (caddr (transicion color-actual cambio-color)) )
304 316 - (format stream "~% --- Fin del Informe ---% ")
305 317 + ;pasandole login para ver en pantalla
306 318 (loginlights color-actual cambio-color)
307 319 )
308 - ;pasandole login para ver en pantalla y luego la impresion dentro del archivo
```

En este commit se corrigió el bug de “Fin de informe”, ya que cada vez que ocurría una llamada con la función logging, se imprimía el texto y este se repetía varias veces. Para evitar esto, se creó una función distinta para que, luego de todas las llamadas de logging, se imprimiera el mensaje de fin de informe al final..

Además, se realizó un cambio con respecto a la función logging, para que en vez de imprimir el tiempo actual, imprimiera el tiempo Unix que inserta el usuario. Por lo que este cambio en logging afecta indirectamente a la función informe

Commit 70c945f:

```

302 + ;;; -----
21 303 (defun distribucion-Time (unix)
22 304   (if (zerop (mod unix 225)) "| 40.0% rojo| 1.33% rojo-intermitente| 53.33% verde| 1.33% verde-intermitente| 2.66% amarillo| 1.33% amarillo intermitente|"
23 305   (format nil "~%| ~%X rojo| ~%X rojo-intermitente| ~%X verde | ~%X verde-intermitente| ~%X amarillo| ~%X amarillo-intermitente|")
24 - (nth 0 (calcular-Porcentajes (calcular-Resto-Ini (mod unix 225)) (Calcular-Resto-Fin (mod (- 3600 (- 225 (mod unix 225))) 225))))
25 - (nth 1 (calcular-Porcentajes (calcular-Resto-Ini (mod unix 225)) (Calcular-Resto-Fin (mod (- 3600 (- 225 (mod unix 225))) 225))))
26 - (nth 2 (calcular-Porcentajes (calcular-Resto-Ini (mod unix 225)) (Calcular-Resto-Fin (mod (- 3600 (- 225 (mod unix 225))) 225))))
27 - (nth 3 (calcular-Porcentajes (calcular-Resto-Ini (mod unix 225)) (Calcular-Resto-Fin (mod (- 3600 (- 225 (mod unix 225))) 225))))
28 - (nth 4 (calcular-Porcentajes (calcular-Resto-Ini (mod unix 225)) (Calcular-Resto-Fin (mod (- 3600 (- 225 (mod unix 225))) 225))))
29 - (nth 5 (calcular-Porcentajes (calcular-Resto-Ini (mod unix 225)) (Calcular-Resto-Fin (mod (- 3600 (- 225 (mod unix 225))) 225))))
30 - )
306 + (first (calcular-Porcentajes (calcular-Resto-Ini (mod unix 225)) (Calcular-Resto-Fin (mod (- 3600 (- 225 (mod unix 225))) 225))))
307 + (second (calcular-Porcentajes (calcular-Resto-Ini (mod unix 225)) (Calcular-Resto-Fin (mod (- 3600 (- 225 (mod unix 225))) 225))))
308 + (third (calcular-Porcentajes (calcular-Resto-Ini (mod unix 225)) (Calcular-Resto-Fin (mod (- 3600 (- 225 (mod unix 225))) 225))))
309 + (fourth (calcular-Porcentajes (calcular-Resto-Ini (mod unix 225)) (Calcular-Resto-Fin (mod (- 3600 (- 225 (mod unix 225))) 225))))
310 + (fifth (calcular-Porcentajes (calcular-Resto-Ini (mod unix 225)) (Calcular-Resto-Fin (mod (- 3600 (- 225 (mod unix 225))) 225))))
311 + (sixth (calcular-Porcentajes (calcular-Resto-Ini (mod unix 225)) (Calcular-Resto-Fin (mod (- 3600 (- 225 (mod unix 225))) 225))))
31 312 )

```

En este commit, se corrige la función (distribución-time) y se hace cambios reemplazando las funciones predefinidas nth por las de first, second y third, para extraer elementos de esa posición específica en la memoria, esto se hace para respetar las reglas de estilo de LISP.

Commit 42b4a0a

```

193 193 IMPACTO: No destructiva
194 194 -----|#
195 195
196 196 #|(defun calcularRestoFin (restoFin)
197 197   (cond
198 - ((<= 0 restoFin 89) (list (- 89 restoFin) 3 117 3 3 3)) ;(- 86 restoFin) --> indica lo consumido por rojo
199 - ((<= 87 restoFin 90) (list 0 (- 90 restoFin) 117 3 3 3)) ;(- 90 restoFin) --> indica lo consumido por el rojo-intermitente
200 - ((<= 91 restoFin 206) (list 0 0 (- 206 restoFin) 3 3 3)) ;(- 206 restoFin) --> indica lo consumido por verde
201 - ((<= 207 restoFin 209) (list 0 0 0 (- 209 restoFin) 3 3)) ;(- 209 restoFin) --> indica lo consumido por verde-intermitente
202 - ((<= 209 restoFin 212) (list 0 0 0 0 (- 212 restoFin) 3)) ;(- 212 restoFin) --> indica lo consumido por amarillo
203 - ((<= 212 restoFin 215) (list 0 0 0 0 0 (- 215 restoFin))) ;(- 215 restoFin) --> indica lo consumido por amarillo-intermitente
198 + ; Al final de la hora es al revés: calculamos cuánto se consumió desde el inicio de ese último ciclo incompleto hasta llegar al segundo 'restoFin'
199 + ((<= 0 restoFin 86) (list restoFin 0 0 0 0 0)) ;(- 86 restoFin) --> indica lo consumido por rojo
200 + ((<= 87 restoFin 89) (list 87 (- restoFin 87) 0 0 0 0)) ;(- 87 restoFin) --> indica lo consumido por el rojo-intermitente
201 + ((<= 90 restoFin 206) (list 87 3 (- restoFin 90) 0 0 0 0)) ;(- 90 restoFin) --> indica lo consumido por verde
202 + ((<= 207 restoFin 209) (list 87 3 117 (- restoFin 207) 0 0)) ;(- 207 restoFin) --> indica lo consumido por verde-intermitente
203 + ((<= 210 restoFin 212) (list 87 3 117 3 (- restoFin 210) 0)) ;(- 210 restoFin) --> indica lo consumido por amarillo
204 + ((<= 213 restoFin 215) (list 87 3 117 3 3 (- restoFin 213))) ;(- 213 restoFin) --> indica lo consumido por amarillo-intermitente
204 205 )
205 206 )|#
206 207 (defun calcularRestoFin (restoFin)

```

En este commit se hicieron cambios a la función calcularestofin, es un error aritmético debido a que el minuendo (el cual era el valor máximo que podía tomar los distintos colores) era siempre mayor al cálculo de restos, ocasionando que el resultado sea negativo. Como se puede observar en la imagen, en la franja verde, se hizo la debida corrección, cambiando el minuendo como el restoFin de la operación y haciendo corrección sobre los valores del sustraendo, tomando los mínimos de cada color (Ej: Antes si el rojo iba de 0 a 89, utilizaba el de 89 directamente como minuendo, ahora con la debida modificación no hace falta modificaciones en ese intervalo, pero si en el resto).

```

(defun calcular-resto-fin (resto-fin)
  (cond ; (89 92 212 215 221 224)
    ; Al final de la hora es al revés: calculamos cuánto se consumió desde el inicio de ese último ciclo incompleto
    ; hasta llegar al segundo 'resto-fin'
    ((<= 0 resto-fin 89) (list resto-fin 0 0 0 0 0))
    ; resto-fin --> indica lo consumido por rojo
    ((<= 90 resto-fin 92) (list 90 (- resto-fin 90) 0 0 0 0))
    ; (- resto-fin 90) --> indica lo consumido por el rojo-intermitente
    ((<= 93 resto-fin 212) (list 90 3 (- resto-fin 93) 0 0 0))
    ; (- resto-fin 93) --> indica lo consumido por verde
    ((<= 213 resto-fin 215) (list 90 3 120 (- resto-fin 213) 0 0))
    ; (- resto-fin 213) --> indica lo consumido por verde-intermitente
    ((<= 216 resto-fin 221) (list 90 3 120 3 (- resto-fin 216) 0))
    ; (- resto-fin 221) --> indica lo consumido por amarillo
    (t (list 90 3 120 3 6 (- resto-fin 222))))
    ; (- resto-fin 222) --> indica lo consumido por amarillo-intermitente
  )
)

```

Luego hubo otro error conceptual, pero afectó a todas las funciones incluida esta, con respecto a los tiempos de ciclo, nuestro equipo decidió agregar los 9 segundos de intermitencia a cada una de las funciones, incluida la de calcularresto-fin, (que ahora se llama calcular-resto-fin). Lo hicimos por una mala interpretación de las consignas. En la imagen se puede ver la función actual

Commit: fc95dc7

```

78 78 ;desactualizado, restamos al tiempo el cual esta dado desde 1970 unos 70 años para que se sincronen correctamente.
79 79
80 - (defun logginlights (color-actual cambio-color)
81 -   (format t "Tiempo ~A: la luz ah cambiado de color ~A a ~A"
82 -     (local-time:format-timestring nil (local-time:now) :format '(:year 4) "-" (:month 2) "-" (:day 2) " " (:hour 2) ":" (:min 2))))
80 + (defun logginlights (color-actual cambio-color unixtemp)
81 +   (format t "Tiempo ~A: la luz ah cambiado de color ~A a ~A%"
82 +     (local-time:format-timestring nil (local-time:unix-to-timestamp unixtemp) :format '(:year 4) "-" (:month 2) "-" (:day 2) " " (:hour 2) ":" (:min 2))))
83 83   (car (transicion color-actual cambio-color)) (caddr (transicion color-actual cambio-color))
84 84   ) )
85 85

```

Como se mencionó anteriormente, en este commit hubo cambios en la función loggings, debido a que anteriormente la función imprimía el tiempo de forma actual, por lo que para nosotros hacía que el registro fuera menos exacto. En cambio, decidimos agregar un tercer parámetro que reciba el tiempo en formato Unix, para así ser más precisos con los registros del tiempo y así en las llamadas de esta función, se pudiera insertar el tiempo donde ocurrieron los cambios realizados por el semáforo.

Commit a314234:

```
10 (defun transicion (color-actual cambiar-a)
11   (cond
12     - ((and (equal color-actual 'en-verde) (equal cambiar-a 'cambiar-a-amarillo)) (list color-actual 'verde-intermitente "CAMBIAR-A-AMARILLO" ))
13     - ((and (equal color-actual 'en-amarillo) (equal cambiar-a 'cambiar-a-rojo)) (list color-actual 'amarillo-intermitente "CAMBIAR-A-ROJO"))
14     - ((and (equal color-actual 'en-rojo) (equal cambiar-a 'cambiar-a-verde)) (list color-actual 'rojo-intermitente "CAMBIAR-A-VERDE" ))
15     - (t (list color-actual 'accion-por-defecto ))
16     + ((and (equal color-actual 'en-verde-intermitente) (equal cambiar-a 'cambiar-a-amarillo)) (list color-actual "CAMBIAR-A-AMARILLO" ))
17     + ((and (equal color-actual 'en-amarillo-intermitente) (equal cambiar-a 'cambiar-a-rojo)) (list color-actual "CAMBIAR-A-ROJO"))
18     + ((and (equal color-actual 'en-rojo-intermitente) (equal cambiar-a 'cambiar-a-verde)) (list color-actual "CAMBIAR-A-VERDE" ))
19     + ((and (equal color-actual 'en-verde) (equal cambiar-a 'cambiar-a-verde-intermitente)) (list color-actual "CAMBIAR-A-VERDE-INTERMITENTE"))
20     + ((and (equal color-actual 'en-amarillo) (equal cambiar-a 'cambiar-a-amarillo-intermitente)) (list color-actual "CAMBIAR-A-AMARILLO-INTERMITENTE"))
21     + ((and (equal color-actual 'en-rojo) (equal cambiar-a 'cambiar-a-rojo-intermitente)) (list color-actual "CAMBIAR-A-ROJO-INTERMITENTE"))
22     + (t (list color-actual 'accion-por-defecto))
23   )
24 )
```

En este commit, hubo cambios con respecto a las transiciones de la funcion transicion y se agregaron mas casos de retorno para que esten todos los colores y sus intermitencias (ROJO-VERDE-AMARILLO-ROJOINTERMITENTE-VERDEINTERMITENTE-AMARILLOINTERMITENTE)

Justificacion (Fase 2)

Fase 2:

Se decidió utilizar la librería (local-time) del gestor de paquetes de quicklisp, debido a la facilidad que esta nos resultó para implementarla en las funciones informe y loginLights a comparación de la biblioteca (cl-json), que para los integrantes, resultó algo tedioso y confuso, combinar el archivo core.lisp junto con el archivo config.json

Analisis (Fase 3)

Presentación breve del lenguaje Scala:

Scala es un lenguaje de programación multiparadigma que combina características de la programación orientada a objetos y la programación funcional. Fue diseñado por Martin Odersky y se ejecuta sobre la máquina virtual de java(JVM), lo que permite aprovechar el ecosistema de Java y sus bibliotecas. Su objetivo es ofrecer un lenguaje expresivo, conciso y seguro para el desarrollo de aplicaciones de distintos tamaños.

Scala se utiliza principalmente en áreas de Big Data, sistemas distribuidos, desarrollo backend y servicios financieros. La fundación Scala destaca que el lenguaje es utilizado para operar aplicaciones distribuidas, pipelines de datos y sistemas críticos en sectores regulados como finanzas y servicios públicos.

```
1  /* funcion: transicion */
2  def transicion(colorActual: String, colorACambiar: String): List[String] = { // declare la funcion, como parametro
3    (colorActual, colorACambiar) match { // funciona como un cond, donde los 2 parametros se comparan en cada caso
4      case ("en-rojo", "en-rojo-intermitente") => List(colorActual, "cambiar-a-rojo-intermitente")
5      // caso 1. => compara los parametros string, cumple con el caso muestra una lista ("en-rojo" "en-rojo-intermitente")
6      case ("en-rojo-intermitente", "en-verde") => List(colorActual, "cambiar-a-verde")
7      // caso 2. => compara los parametros string, cumple con el caso muestra una lista ("en-rojo-intermitente") ("en-verde")
8      case ("en-verde", "en-verde-intermitente") => List(colorActual, "cambiar-a-verde-intermitente")
9      // caso 3. => compara los parametros string, cumple con el caso muestra una lista ("en-verde") ("en-verde-intermitente")
10     case ("en-verde-intermitente", "en-amarillo") => List(colorActual, "cambiar-a-amarillo")
11     // caso 4. => compara los parametros string, cumple con el caso muestra una lista ("en-verde-intermitente") ("en-amarillo")
12     case ("en-amarillo", "en-amarillo-intermitente") => List(colorActual, "cambiar-a-amarillo-intermitente")
13     // caso 5. => compara los parametros string, cumple con el caso muestra una lista ("en-amarillo") ("en-amarillo-intermitente")
14     case ("en-amarillo-intermitente", "en-rojo") => List(colorActual, "cambiar-a-rojo")
15     // caso 6. => compara los parametros string, cumple con el caso muestra una lista ("en-verde") ("en-verde-intermitente")
16     case _ => List(colorActual, "accion-por-defecto")
17     // caso 7. => indica que si no se cumple con las condiciones anteriores, este devuelve el color actual y accion
18   }
19 }
20 // actualizado con intermitentes
```

Función de transición implementada en Scala

La función transición recibe como parámetros colorActual y ColorACambiar(ambos declarados como string) y devuelve una lista string. Dentro de la función utilizamos un match el cual recibirá nuestros parámetros, los cuales en cada caso serán comparados con los colores del semáforo(los cuales deben cumplir con el ciclo rojo→verde→amarillo), si cumple en algún caso se ejecutará y retornará la lista de ese caso, si no cumple con el ciclo del semáforo se retornará una lista con el color actual y el string "accion-por-defecto"

```
43
44  /*funcion timer*/
45  def timer(timestamp: Float): String = { //funcion timer que recibe el parametro timestamp
46    (timestamp % 225.0f) match { //declaro el match con el parametro timestamp
47      // Aplicamos el módulo 225.0f para limpiar los ciclos completos, siempre va a valer entre 0 y 224
48      case time if time >= 0.0f && time <= 89.0f => "en-rojo"
49      // caso 1. => comparamos si esta dentro del rango para ser rojo
50      case time if time >= 90.0f && time <= 92.0f => "en-rojo-intermitente"
51      // caso 2. => comparamos si esta dentro del rango para ser rojo-intermitente
52      case time if time >= 93.0f && time <= 112.0f => "en-verde"
53      // caso 3. => comparamos si esta dentro del rango para ser verde
54      case time if time >= 113.0f && time <= 115.0f => "en-verde-intermitente"
55      // caso 4. => comparamos si esta dentro del rango para ser verde-intermitente
56      case time if time >= 116.0f && time <= 121.0f => "en-amarillo"
57      // caso 5. => comparamos si esta dentro del rango para ser amarillo
58      case time if time >= 122.0f && time <= 124.0f => "en-amarillo-intermitente"
59      // caso 6. => comparamos si esta dentro del rango para ser amarillo-intermitente
60    }
61  }
```

Función de timer implementada en Scala

La función timer recibe como parametro timestap (el cual se declara como float) y retorna un string.

Dentro de la función utilizamos un match el cual recibe timestap % 225, es decir que el match utilizaria el resto de la división entre timestap y 225, donde en cada caso se vera si ese resto , el cual en los caso se los nombra como time cumple con algún rango determinado de los casos, si es asi se ejecuta y se muestra el string.

Bibliografias

Bibliografía utilizada:

<https://docs.scala-lang.org>

<https://docs.scala-lang.org/scala3/book/introduction.html>

<https://docs.scala-lang.org/scala3/book/taste-vars-data-types.html>

<https://docs.scala-lang.org/scala3/book/methods-most.html>

<https://docs.scala-lang.org/scala3/book/control-structures.html>

<https://docs.scala-lang.org/tour/pattern-matching.html>

<https://docs.scala-lang.org/scala3/book/tuples.html>

<https://docs.scala-lang.org/overviews/collections-2.13/concrete-immutable-collection-classes.html>

<https://docs.scala-lang.org/tour/pattern-matching.html>

<https://scala-lang.org/files/archive/spec/3.7/06-expressions.html>

<https://docs.scala-lang.org/scala3/book/control-structures.html>

<https://www.scala-lang.org/blog/2026/01/27/sta-invests-in-scala.html>

<https://www.scala-lang.org/blog/2026/01/27/sta-invests-in-scala.html>

<https://www.scala-lang.org/blog/2026/01/27/sta-invests-in-scala.html>

Conclusion

Conclusión

La realización de este proyecto nos permitió afianzar conocimientos de programación funcional en Common Lisp, aplicando conceptos vistos durante la cursada a un problema práctico. Durante el desarrollo del sistema de semáforos inteligentes enfrentamos distintos desafíos, tanto de implementación como de lógica, que fueron resueltos mediante pruebas, correcciones y trabajo en equipo.

Además, la investigación sobre Scala nos permitió conocer otro enfoque de programación y comparar sus características con las de Common Lisp. En general, esta experiencia resultó enriquecedora, ya que no solo nos ayudó a mejorar nuestras habilidades técnicas, sino también nuestra capacidad de análisis, depuración y organización de un proyecto de software.

Preguntas teóricas (Fase 3)

Respuestas sobre las preguntas sobre Scala:

1. Scala combina la Programación Orientada a Objetos (POO) con la Funcional.
¿Cómo estructuraron el semáforo? ¿Usaron una clase tradicional, un `object` (Singleton), o `case classes`? Justifiquen desde el diseño funcional.
 2. Comparen la manipulación de listas en Scala (métodos como `.map` o `.filter`) contra las funciones de orden superior de Common Lisp. ¿Cuál resulta más legible y por qué?
-
1. Con respecto a la estructura del semáforo, no utilizamos clases como el `objet` ni `case clases`. El programa se diseñó con funciones puras, en donde cada función responde según sus parámetros de entrada, resolviendo los problemas de manera funcional, devolviendo resultados sin modificar ningún tipo estructuras externas e internas
 2. Investigando sobre el tema, los métodos mencionados de `.map` y `.filter`, es mucho más fácil y resumido de entender que la sintaxis del common lisp. ejemplo:
lisp: (remove-if-not #'(lambda (x) (> x 0)) lista)
scala: lista.filter(x => x > 0)
para alguien que mira estas sintaxis por primera vez, es mucho más fácil descifrar y de entender la sintaxis de scala, siendo más directo respecto a los nombres y funciones. Por otra parte, la ventaja que presentó Scala respecto a lisp para el grupo, fue el gran parecido que tiene con el lenguaje de C + + en su sintaxis y no utilizando un lenguaje polaco como lo es lisp.